Project Report

UNIGRIEVE: Unified Grievance Management System

Submitted by

Rudraksha Bhatnagar

Registration No: 23BCE1366

Video Demonstration :
https://drive.google.com/file/d/1EH8yZNix52hdbK4EewZZA-XGqHMfQggL/view?usp=sharing

1. Problem Statement

In academic institutions, the management of student and staff grievances is a persistent challenge. Traditional grievance handling mechanisms are predominantly manual — relying on handwritten complaint forms, physical submission counters, or basic email-based systems. These approaches suffer from several critical shortcomings:

- Lack of transparency — complainants receive no real-time visibility into the status of their complaints, leading to frustration and repeated follow-ups.
- Inefficient routing — grievances are often misdirected to wrong departments due to manual classification, causing delays of days or even weeks.
- No accountability tracking — there is no systematic mechanism to track officer response times, SLA adherence, or workload distribution.
- Missing historical data — paper-based or unstructured digital systems make it nearly impossible to analyze trends, identify recurring issues, or generate institutional reports.
- Poor user experience — existing web portals (where they exist) are outdated, non-responsive, and do not provide intuitive workflows for submission, tracking, or management.

The absence of an integrated, automated, and role-aware grievance management system results in delayed resolutions, decreased stakeholder satisfaction, and an inability for administration to make data-driven improvements.

UNIGRIEVE addresses these challenges by providing an intelligent, department-oriented, web-based system that automates grievance routing, enforces SLA deadlines, and delivers a modern, role-based user experience for citizens (students/staff), departmental officers, department administrators, and system administrators.

2. Objectives

The primary objectives of the UNIGRIEVE project are:

- Design and develop a full-stack web application for end-to-end grievance lifecycle management — from submission to resolution with feedback.
- Implement intelligent auto-routing that maps grievance categories to the correct department automatically, eliminating manual triage and reducing response latency.
- Provide role-based dashboards offering tailored views and actions for citizens, officers, department administrators, and system administrators.
- Enforce SLA (Service Level Agreement) timelines on each grievance category with automated deadline computation, enabling accountability and performance tracking.
- Enable file attachment support for supporting documents (PDF, JPG, PNG, DOC, DOCX) with secure local storage.
- Support a complete status workflow (Submitted → In Review → In Progress → Awaiting Info → Resolved → Closed / Escalated) with a persistent status timeline and audit trail.
- Build an internal remarks system for officer-to-officer communication on grievance cases, invisible to complainants.
- Provide system administration features including user management (role/department assignment, deactivation), department management, and category configuration.
- Ensure secure authentication using JWT (JSON Web Tokens) with short-lived access tokens and long-lived refresh tokens, plus bcrypt password hashing.
- Deliver a modern, responsive, and aesthetically premium user interface following a structured design system with CSS custom properties and modular component architecture.

3. Methodology / Approach

3.1 Technology Stack

Layer	Technology	Purpose
Backend Runtime	Node.js + Express.js	RESTful API server with MVC architecture
Database	MongoDB + Mongoose	NoSQL document database with embedded arrays for grievance history, remarks, and attachments
Frontend	React.js + Vite	Component-based SPA with fast HMR development workflow
Authentication	JWT (jsonwebtoken) + bcryptjs	Access/refresh token strategy with bcrypt password hashing
File Upload	Multer	Disk-based file storage with extension/size validation
Validation	express-validator	Server-side input validation and sanitization
Routing	React Router v6	Client-side routing with protected route guards
Styling	CSS Modules + Custom Properties	Scoped component styles with a centralized design token system

3.2 Architecture

The project follows a monorepo structure with separate server and client packages:

- Backend: MVC (Model-View-Controller) pattern with Mongoose models, Express controllers, and route files.
- Frontend: Component-based architecture with React Context for global state (AuthContext), CSS Modules for scoped styling, and a centralized design token system (tokens.css).
- API Communication: RESTful endpoints with JSON payloads; FormData for multipart file uploads. Vite proxies /api/* requests to the Express backend during development.

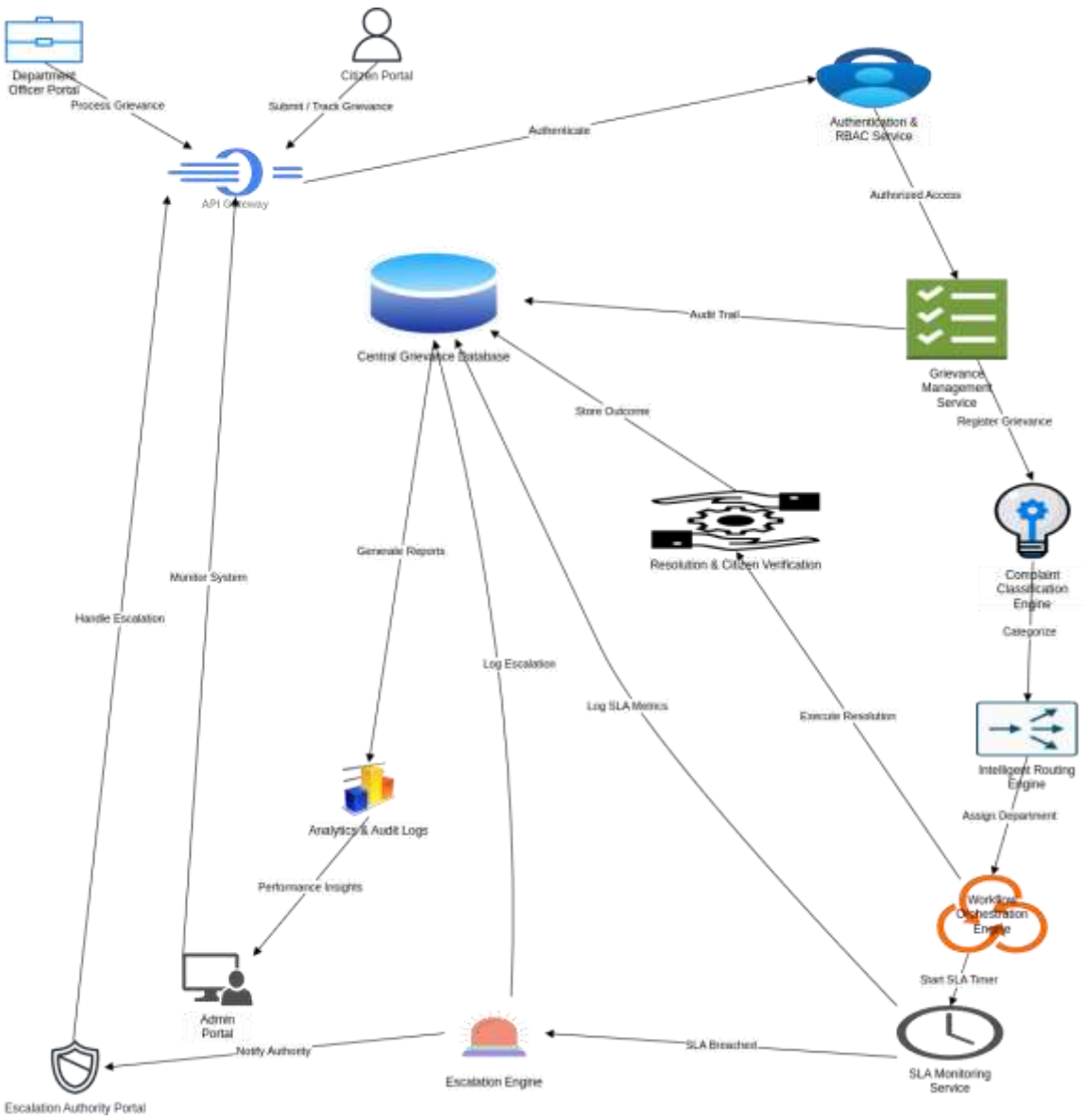


Fig. Architecture Diagram

3.3 Development Methodology

The project is developed in a phased, incremental approach as defined in the PRD (Product Requirements Document v2.0):

Phase 1 — Foundation & Authentication: Project initialization, database connection, User model with bcrypt, JWT auth system (register, login, refresh, getMe), design system CSS, and auth UI pages.

Phase 2 — Grievance Submission & Auto-Routing: Department/Category/Grievance Mongoose models, Multer file upload middleware, auto-routing engine (category →

department mapping), grievance submission form with drag-and-drop file upload, confirmation screen, My Grievances list, and detail page.

Phase 3 — Officer Dashboard & Workflow: Grievance status update workflow, officer assignment, internal remarks system, admin user management, and department/category administration.

Phase 4 (Planned) — Analytics & Notifications: Dashboard statistics with live data, email/in-app notifications for status changes, analytics charts, and SLA breach alerts.

3.4 Security Approach

- Passwords hashed with bcrypt (10 salt rounds) — never stored in plaintext.
- JWT access tokens (short-lived, ~15 min) paired with refresh tokens for session management.
- Role-based access control (RBAC) at both API middleware level (authorizeRoles) and frontend route level (ProtectedRoute).
- File upload validation — extension whitelist (PDF, JPG, PNG, DOC, DOCX), 5MB size limit per file, max 5 files per request.
- Internal remarks on grievances hidden from citizen role — data-level access control.

4. Work Completed So Far (50% Implementation)

As of March 2026, Phases 1 through 3 have been fully implemented and verified. This constitutes approximately 50% of the overall system functionality planned in the PRD. Below is a detailed breakdown of the completed work:

4.1 Phase 1 — Foundation & Authentication (100% Complete)

- Project initialized with Node.js/Express backend and React/Vite frontend in monorepo structure.
- MongoDB connected via Mongoose with environment-variable-driven configuration.
- User model created with schema validation, bcrypt pre-save password hashing, role enum (citizen, officer, deptAdmin, sysAdmin), departmentId reference, and isActive flag.

- Four authentication API endpoints implemented:
    - POST /api/auth/register — user registration with express-validator input validation.
    - POST /api/auth/login — credential verification with JWT access + refresh token issuance.
    - POST /api/auth/refresh — silent token refresh using HTTP-only cookie.
    - GET /api/auth/me — retrieve authenticated user profile.
- Auth middleware (protect) and role-based authorization middleware (authorizeRoles) implemented.
- Global error handler for Mongoose validation errors, duplicate keys, and cast errors.
- Complete design system implemented with CSS custom properties (tokens.css) covering colors, typography, spacing, shadows, and transitions per PRD §6.
- Login and Register pages built with branded UI — glassmorphism cards, gradient background, form validation, password toggle, loading states, and responsive layout.
- Dashboard page with user info card, stats placeholders, and quick action tiles.
- React Router configured with ProtectedRoute component for role-based access.

4.2 Phase 2 — Grievance Submission & Auto-Routing (100% Complete)

- Three new Mongoose models created:
    - Department — name, contactEmail, headOfficerId, isActive flag.
    - Category — name, departmentId (for auto-routing), slaDays (SLA enforcement), description. Compound unique index on name+departmentId.
    - Grievance — grievanceId (GRV-YYMMDD-XXXX format), embedded statusHistory[], remarks[], attachments[], feedback{} subdocuments. Pre-save hook auto-appends status changes to history. Compound indexes for performance.
- Multer middleware configured for local disk storage with extension whitelist and 5MB/5-file limits.
- Auto-routing engine: on grievance submission, the selected category's departmentId automatically assigns the grievance to the correct department, and the SLA deadline is computed from category.slaDays.

- Seed script (seed.js) created with 5 departments and 14 categories for an academic institution (Academic Affairs, Administration, Finance, Hostel, IT).
- Grievance API endpoints:
    - POST /api/grievances — submit with file upload and auto-routing.
    - GET /api/grievances — role-scoped list with pagination (citizen=own, officer=assigned, admin=dept, sysAdmin=all).
    - GET /api/grievances/:id — full detail with populated references.
- Submit Grievance page with category dropdown (live from API), priority pill selector (Low/Medium/High/Critical), drag-and-drop file upload zone, form validation, and FormData submission.
- Confirmation page showing generated GRV-ID, assigned department, priority, and SLA deadline.
- My Grievances page with status filter pills, color-coded cards with status badges, pagination, and empty state.
- Grievance Detail page with info card, status timeline (from embedded statusHistory), and attachments list.
- Sidebar navigation component with role-aware links — citizens see submit/track, officers see assigned, admins see management sections. Responsive collapse to icon-only on mobile.

4.3 Phase 3 — Officer Dashboard & Workflow (100% Complete)

- Three grievance workflow API endpoints added:
    - PATCH /api/grievances/:id/status — officers/admins update status with optional note; triggers pre-save hook to log in statusHistory.
    - PATCH /api/grievances/:id/assign — deptAdmin assigns officer; auto-transitions status from Submitted to "In Review".
    - POST /api/grievances/:id/remarks — officers add internal remarks (hidden from citizens via data-level access control).
- Admin API controller created with four endpoints:
    - GET /api/admin/users — list all users with search, role, and department filters.

    - GET /api/admin/users/officers — officer list for assignment dropdown (dept-scoped for deptAdmin).
    - PATCH /api/admin/users/:id — update user role and department assignment.
    - DELETE /api/admin/users/:id — soft-delete (deactivate) a user.
- Assigned Grievances page — officer view with status filters, linking to the manage page.
- Manage Grievance page — full workflow panel: status update dropdown with note field, officer assignment (deptAdmin+), internal remarks system with chronological listing, status timeline, and attachments.
- Admin Users page — data table with search input, role filter dropdown, edit modal for changing user role/department, and deactivate action with confirmation.
- Admin Departments page — card grid showing departments with their categories and SLA days, create/edit modal.
- All Phase 3 routes protected with role guards via ProtectedRoute component.

4.4 API Endpoints Summary

Method	Endpoint	Access
POST	/api/auth/register	Public
POST	/api/auth/login	Public
POST	/api/auth/refresh	Public
GET	/api/auth/me	Authenticated
POST	/api/grievances	Authenticated
GET	/api/grievances	Authenticated
GET	/api/grievances/:id	Authenticated
GET	/api/departments	Authenticated
GET	/api/categories	Authenticated
PATCH	/api/grievances/:id/status	Officer
PATCH	/api/grievances/:id/assign	DeptAdmin
POST	/api/grievances/:id/remarks	Officer
GET	/api/admin/users	SysAdmin
PATCH	/api/admin/users/:id	SysAdmin
DELETE	/api/admin/users/:id	SysAdmin

4.5 Verification Results

- Backend server starts successfully and connects to MongoDB.
- Seed script creates 5 departments and 14 categories without errors.
- Frontend build (Vite production build) completes successfully — 69.29 kB bundled in 1.43 seconds with zero compilation errors.
- All React components import and render correctly with proper CSS module scoping.
- Role-guarded routes function correctly — unauthorized users are redirected.

4.7 Remaining Work (Phase 4 — Planned)

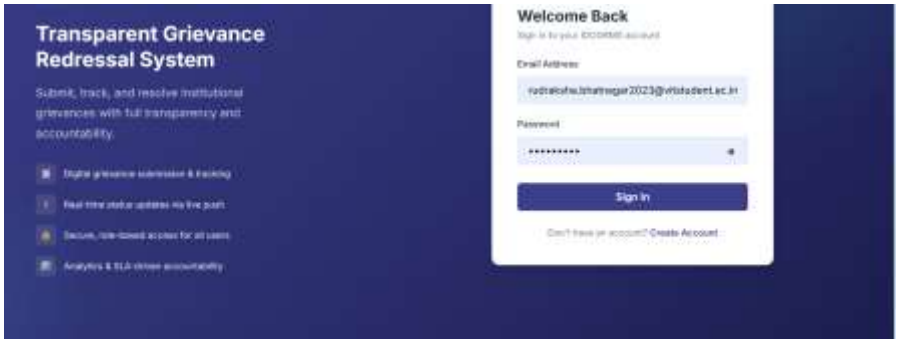
The remaining 50% of the project will be completed in Phase 4, which focuses on:

- Dashboard analytics — live grievance statistics (total, pending, resolved, escalated) populated from API aggregation queries.
- SLA breach detection and alerts — automated identification of grievances exceeding their SLA deadline.
- Notification system — email and/or in-app notifications when grievance status changes, new assignments occur, or SLA deadlines approach.
- Analytics charts — visual data representations (bar charts, pie charts) showing department performance, category distribution, and resolution trends.
- Citizen feedback — post-resolution rating and comment system to measure satisfaction.
- Export and reporting — ability to export grievance data as CSV/PDF for institutional reporting.

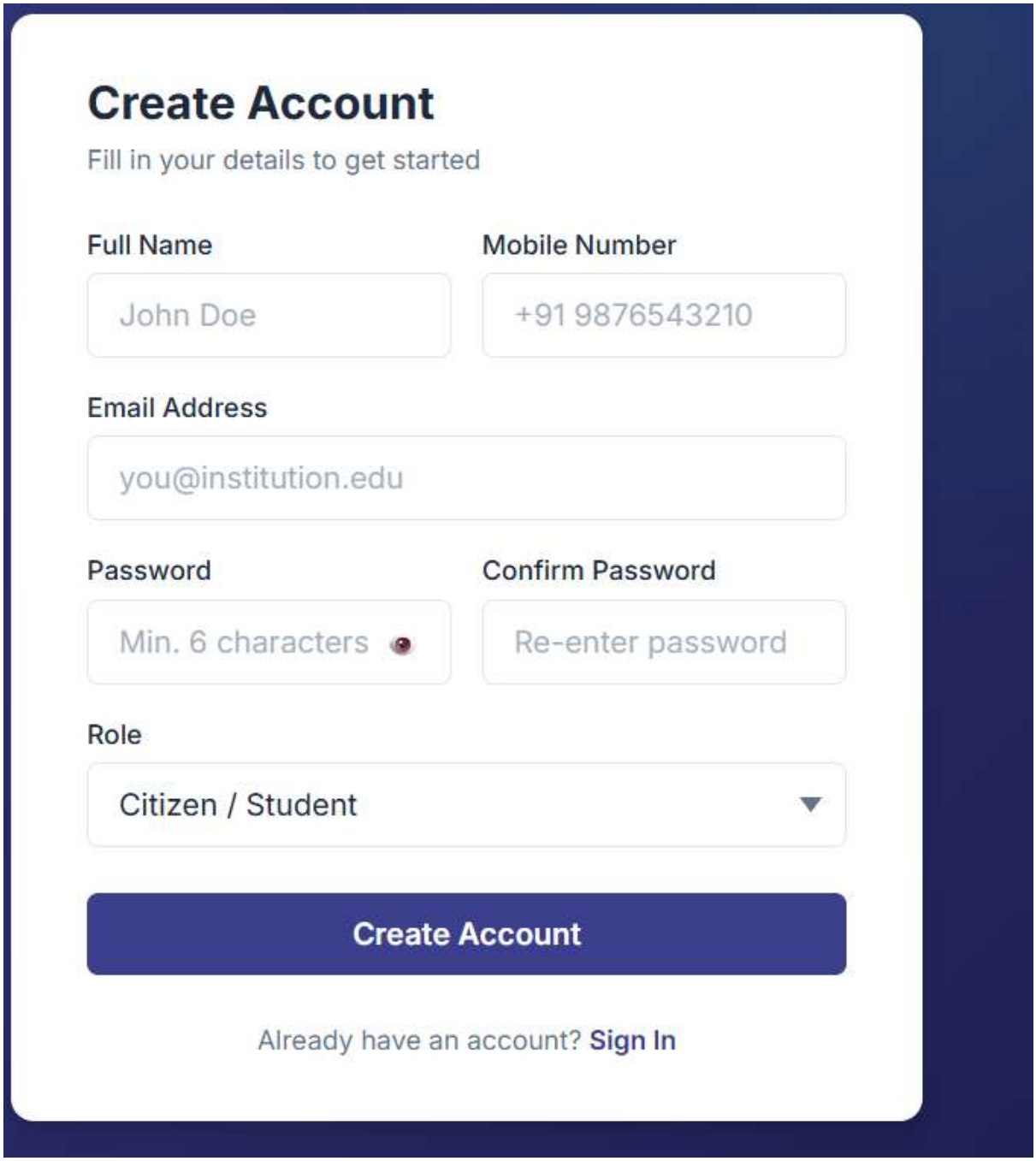
5. Screenshots (Proof of work)

Github Link: https://github.com/Rudraksha-Bhatnagar/23BCE1366_UniGriev/tree/main/project

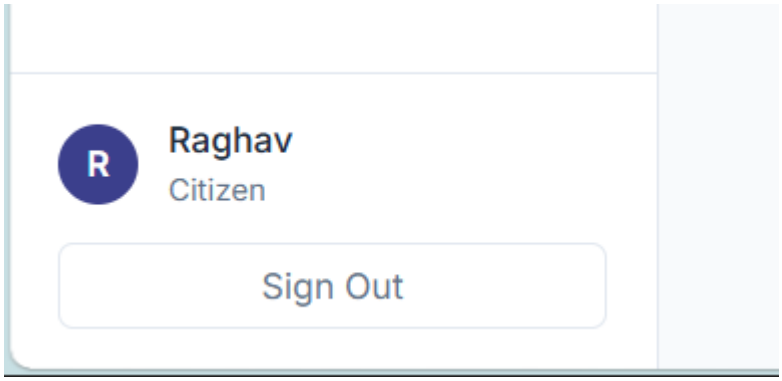
Login:



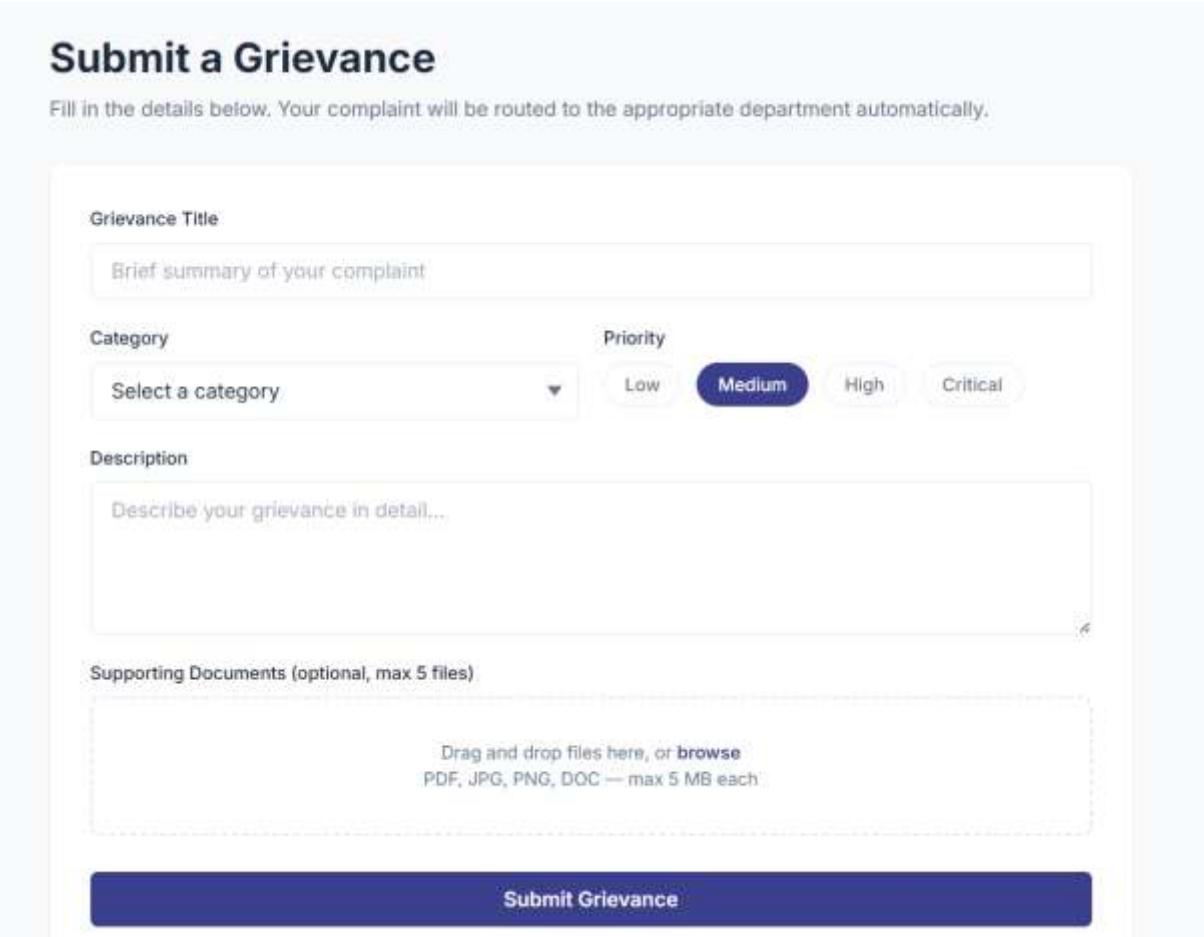
Sign Up:



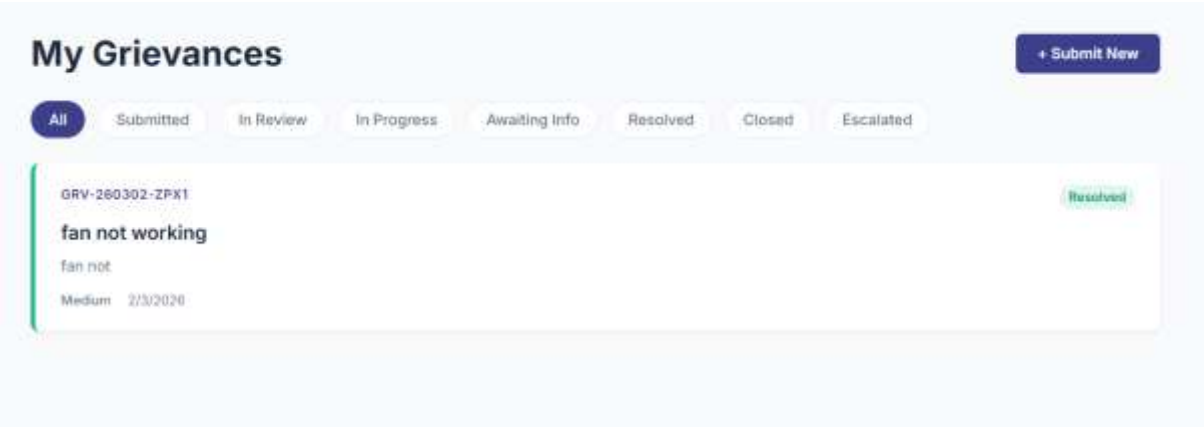
Citizen Portal:



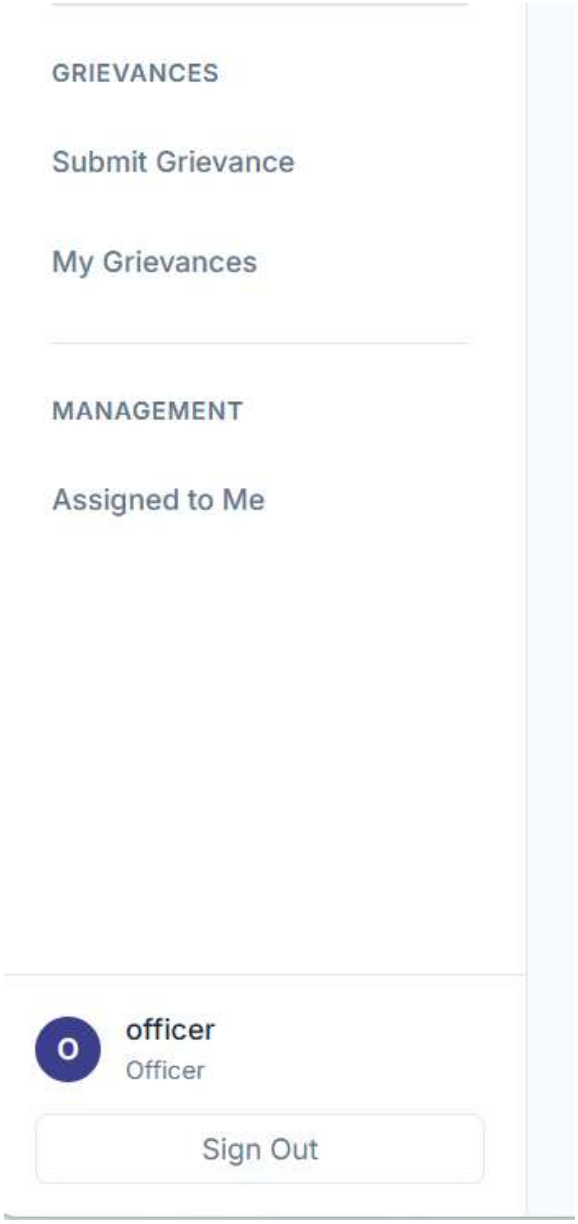
Submit Grievance



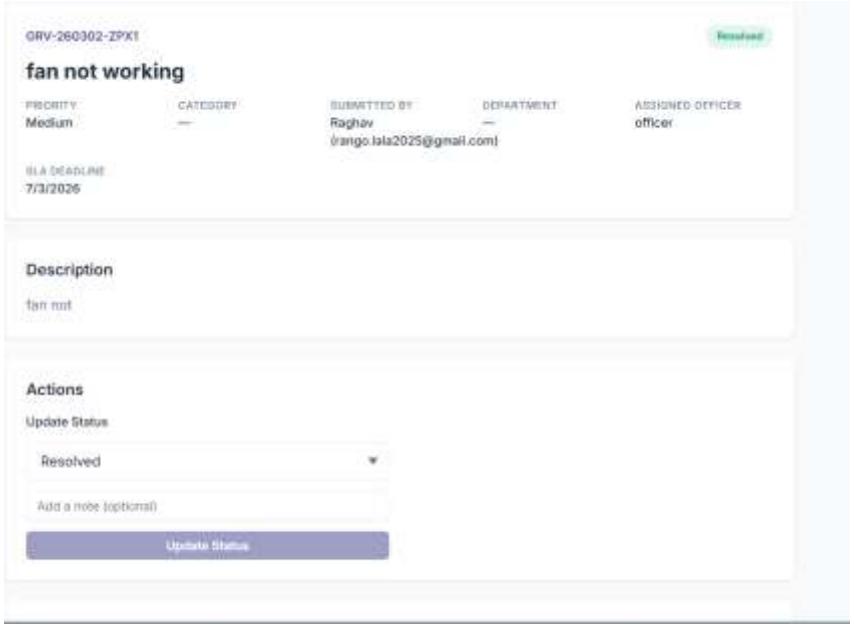
View Grievance



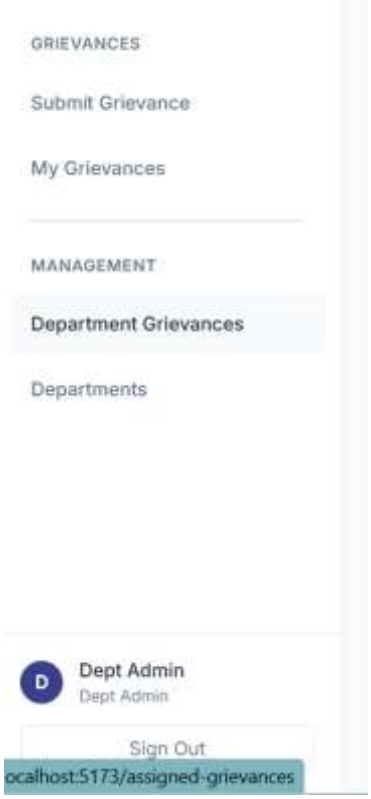
Officer Portal



Grievances Assigned to officer



Department Admin Portal:



View Grievance and Assign Officer with SLA Guide Line

GRV-260302-ZPX1

Resolved

fan not working

PRIORITY	CATEGORY	SUBMITTED BY	DEPARTMENT	ASSIGNED OFFICER
Medium	—	Raghav (rango.lala2025@gmail.com)	—	officer

SLA DEADLINE
7/3/2026

Description

fan not

Actions

Update Status

Resolved

Add a note (optional)

Update Status

Assign Officer

Select an officer

Assign

Status of Grievance:

Internal Remarks

Write an internal note...

Add

No remarks yet

Status Timeline

- **Resolved**
  officer - 3/3/2026, 10:22:23 pm
  Status changed to Resolved
- **In Review**
  Dept Admin - 3/3/2026, 10:20:53 pm
  Status changed to In Review
- **Submitted**
  Raghav - 2/3/2026, 9:34:51 pm
- **Submitted**
  Raghav - 2/3/2026, 9:34:51 pm
  Grievance submitted